



Centre de formation **MAX_ADIS**

Crée et contrôle tes propres systèmes intelligents



Rapport de Projet

Thème :

***Régulateur thermique connecté avec supervision et
commande manuelle du ventilateur***



Réalisé par :

- *FADONUGBO Anselme*
- *KINDOMISSI Achille*
- *HOUNKOKOE Trifène*

Edition 2026

1. Présentation générale du projet

Avec les progrès de l'électronique et de l'Internet des Objets (IoT), il devient possible de surveiller et de contrôler différents équipements à distance. La régulation de la température est un besoin présent dans plusieurs domaines tels que les habitations, les salles informatiques, les entreprises, et les laboratoires. Notre projet consiste à concevoir un système capable de mesurer la température ambiante grâce à un capteur DHT11, d'afficher cette température sur une application mobile via Bluetooth et de permettre la commande manuelle d'un ventilateur. Le système est basé sur une carte Arduino Uno qui traite les informations du capteur et pilote un relais commandant un ventilateur. Une application mobile développée avec MIT App Inventor permet à l'utilisateur de superviser la température en temps réel et d'allumer ou d'éteindre le ventilateur.

2. Problématique du projet

Dans de nombreux environnements, la température peut dépasser les limites acceptables et provoquer :

- une diminution des performances des équipements ;
- une surchauffe des composants électroniques ;
- un inconfort des utilisateurs ;
- des pertes matérielles.

Les systèmes classiques nécessitent souvent une intervention humaine permanente.

La question principale est donc :

“ Comment réaliser un système simple, économique et connecté permettant de surveiller la température en temps réel et de commander facilement un ventilateur à distance ? ”

3. Objectifs du projet

Objectif général

Concevoir un système de régulation thermique connecté permettant la supervision de la température et la commande manuelle d'un ventilateur.

Objectifs spécifiques

- Mesurer la température ambiante ;
- Afficher la température sur une application mobile ;
- Transmettre les données par Bluetooth ;
- Commander un ventilateur depuis un smartphone ;

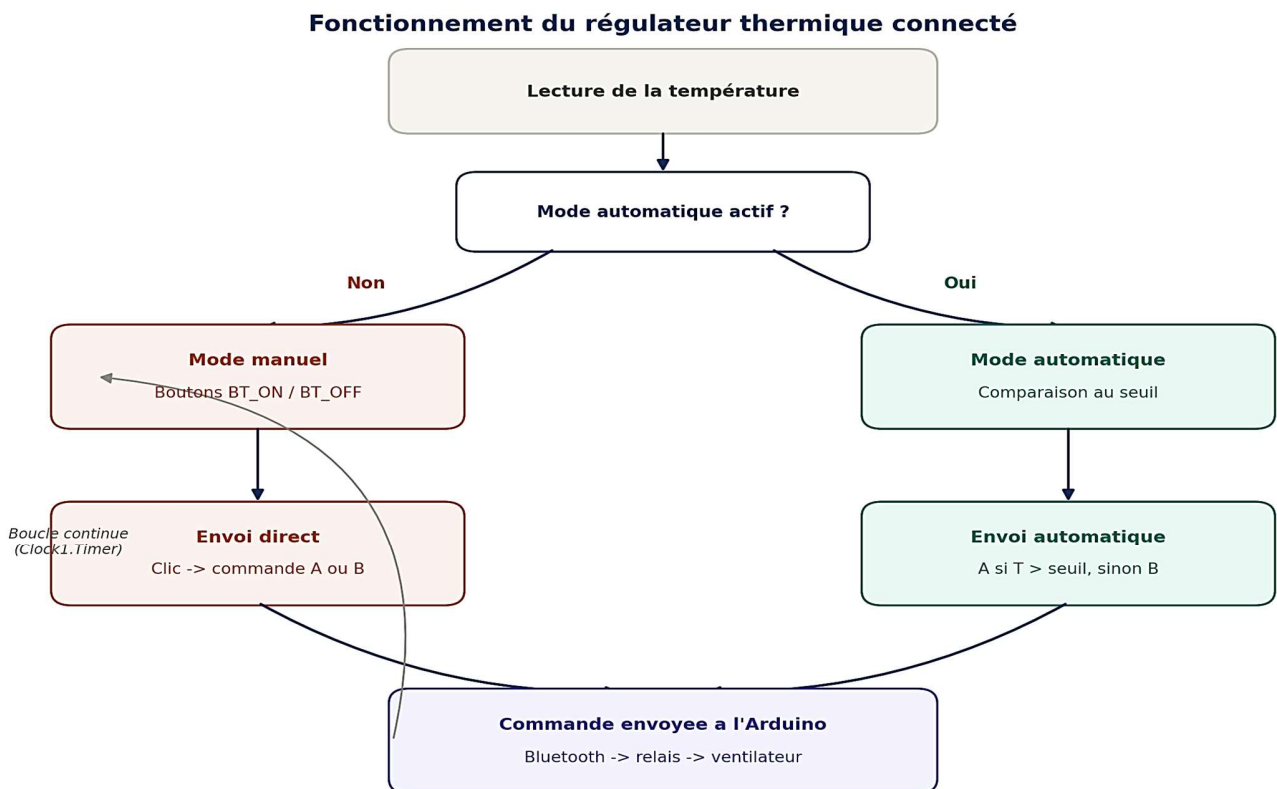
- Assurer un fonctionnement fiable du système.

4. *Fonctionnement du système*

Le système fonctionne selon les étapes suivantes :

1. Le capteur DHT11 mesure la température ambiante.
2. La carte Arduino Uno récupère cette information.
3. La température est envoyée au module Bluetooth HC-05.
4. Le smartphone reçoit la température et l'affiche sur l'application en temps réel.
5. Lorsque l'utilisateur appuie sur le bouton **ON**, l'application envoie une commande Bluetooth.
6. L'Arduino reçoit cette commande.
7. L'Arduino active le relais.
8. Le relais alimente le moteur à courant continu qui entraîne l'hélice du ventilateur.
9. Lorsque l'utilisateur appuie sur **OFF**, le relais se désactive et le ventilateur s'arrête.

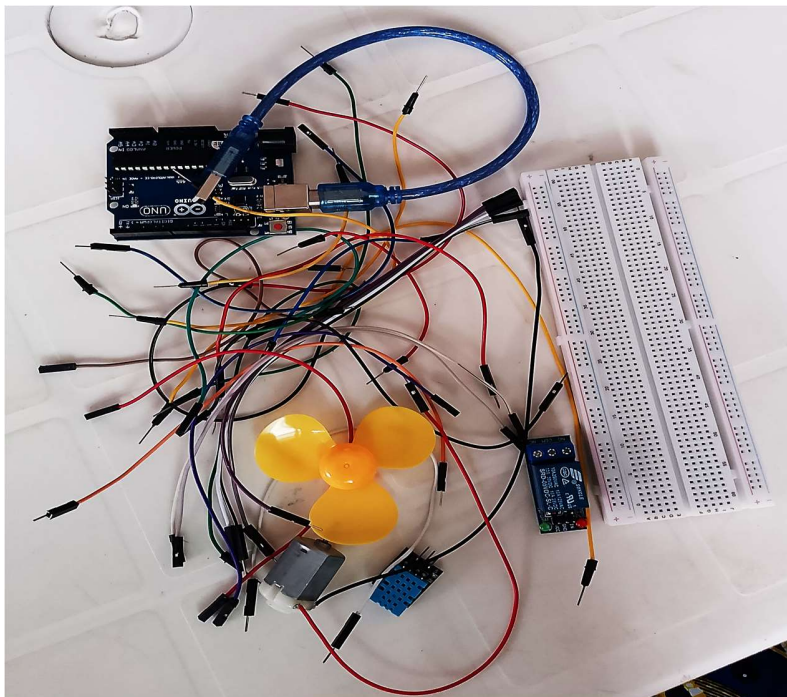
Schéma de fonctionnement du système



5. Liste des matériels utilisés et leur rôle

Matériel	Rôle
Arduino Uno	Traite les données et commande l'ensemble du système.
Capteur DHT11	Mesure la température ambiante.
Module Bluetooth HC-05	Assure la communication entre l'Arduino et le smartphone.
Module relais 1 canal	Permet de commander le ventilateur sans surcharger l'Arduino.
Ventilateur (hélice + moteur CC)	Refroidit l'environnement lorsque l'utilisateur l'active.
Breadboard	Réalisation du montage sans soudure.
Fils Jumpers	Assurent les différentes connexions électriques.
Alimentation	Fournit l'énergie nécessaire au système.
Câble USB Arduino	Permet la programmation de l'Arduino et son alimentation depuis un ordinateur.
Smartphone Android	Sert d'interface de supervision et de commande.

Photo des matériels



6. Programmation

✓ *Langage utilisé :*

Le projet a été programmé en C/C++ (Arduino).

✓ *Environnement de développement (IDE) :*

Le programme a été développé avec : **Arduino IDE**

✓ *Application mobile :*

L'application de supervision nommée **ThermoControl BT** a été réalisée avec : **MIT App Inventor**. Elle permet :

- d'afficher la température en temps réel ;
- de connecter le Bluetooth ;
- d'allumer le ventilateur ;
- d'éteindre le ventilateur.

✓ *Code source*

```
#include "DHT.h"           //Bibliothèque du capteur DHT11
#include <SoftwareSerial.h>  //Bibliothèque du module bluetooth

//Déclaration des variables
SoftwareSerial mySerial(10, 11); // RX, TX vers HC-05
const int DHTPIN = 2;
const int DHTTYPE = DHT11;
const int pinRelais = 3;
DHT dht(DHTPIN, DHTTYPE);
bool modeAuto;
float seuil;

unsigned long dernierEnvoi = 0;
const unsigned long intervalleEnvoi = 1000; // 1000 millisecondes, respecte le DHT11

//Initialisation des variables
void setup() {
  pinMode(DHTPIN, INPUT);
  pinMode(pinRelais, OUTPUT);
  digitalWrite(pinRelais, HIGH); // état initial : ventilateur éteint
  modeAuto = false;
```

```

seuil = 31.0;          //Valeur par défaut de la température modifiable depuis l'application
dht.begin();
Serial.begin(9600);
mySerial.begin(9600);
}

void loop() {
  /* 1. Envoi périodique de la température sur le Bluetooth
  //millis() renvoie le nombre de millisecondes écoulées depuis le démarrage de l'Arduino.
  Pourquoi pas delay(1000) ? Parce que delay() bloquerait complètement
  le programme pendant 1 secondes, empêchant l'Arduino de recevoir une commande "A" ou "B"
  pendant ce temps. Avec millis(), l'Arduino reste réactif en permanence.
  */
  if (millis() - dernierEnvoi >= intervalleEnvoi) { /*Cette ligne vérifie : "est-ce que 2000 ms se
  sont écoulées depuis le dernier envoi ?" Si oui, on entre dans le bloc.*/
    float t = dht.readTemperature();
    if (!isnan(t)) {
      mySerial.println(t);    // envoie juste le nombre, ex: "25.30"
      Serial.print("Température actuelle: ");
      Serial.println(t);      // pour debug sur le moniteur série USB
      if (modeAuto){
        if (t > seuil){
          digitalWrite(pinRelais, LOW);
        }
        else{
          digitalWrite(pinRelais, HIGH);
        }
      }
    }
    dernierEnvoi = millis();
  }

  // 2. Lecture des commandes manuelles reçues depuis l'app (A = ON, B = OFF)

  if (mySerial.available()) {
    String commande = mySerial.readStringUntil('\n');
    commande.trim();

    if (commande == "M1") {
      modeAuto = true;
    }
    else if (commande == "M0") {

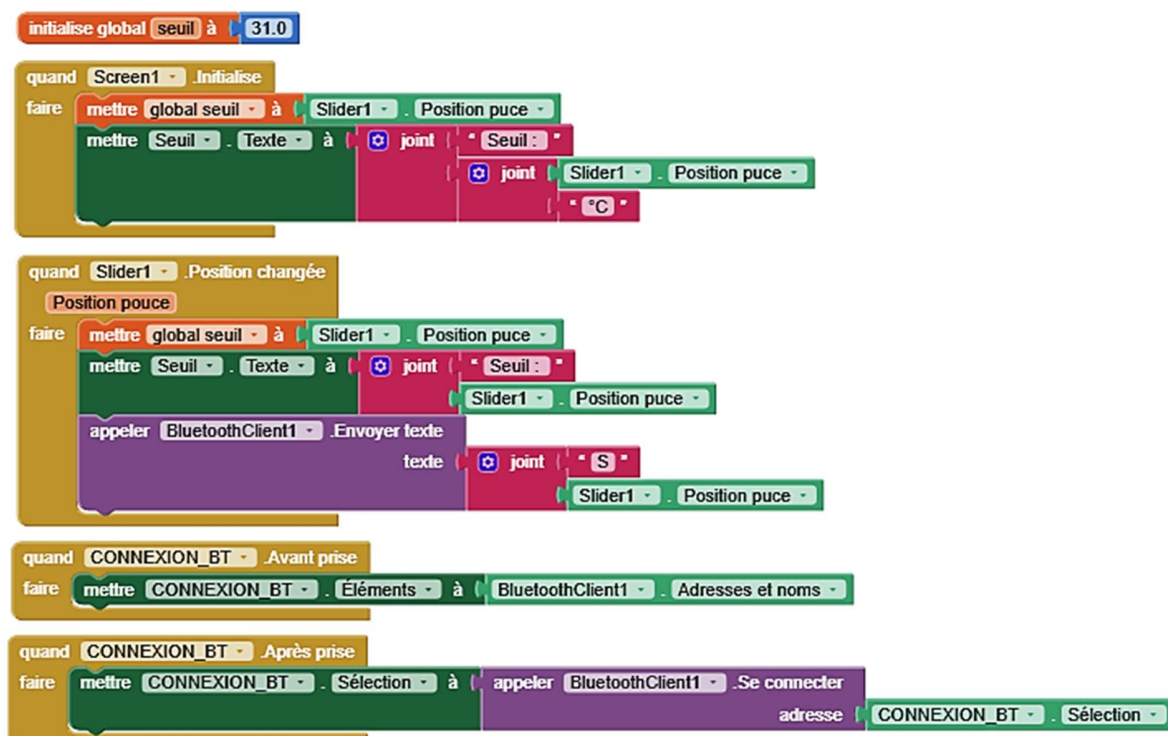
```

```

    modeAuto = false;
}
else if (commande.startsWith("S")) {
    seuil = commande.substring(1).toFloat(); // ex: "S28.5" -> 28.5
    Serial.print("Nouveau seuil reçu");
    Serial.println(seuil); // debug utile pour vérifier
}
else if (commande == "A" && !modeAuto) {
    digitalWrite(pinRelais, LOW); // ventilateur ON
    Serial.println("Commande A reçue -> Ventilateur ON"); // commande manuelle ON
    (uniquement si pas en auto)
}
else if (commande == "B" && !modeAuto) {
    digitalWrite(pinRelais, HIGH); // ventilateur OFF
    Serial.println("Commande B reçue -> Ventilateur OFF"); // commande manuelle OFF
}
}
}
}

```

✓ Blocs MIT App Inventor





7. Cas d'utilisation

Mode Automatique : Le système détecte une chaleur excessive et active le ventilateur sans intervention humaine.

Cas 1 : Consultation de la température

- L'utilisateur ouvre l'application.
- Il connecte le téléphone au module HC-05.
- La température est affichée automatiquement.

Mode Manuel : L'utilisateur constate un inconfort et décide d'activer le ventilateur manuellement via son application, même si la température est en dessous du seuil.

Cas 2 : Mise en marche du ventilateur

- L'utilisateur appuie sur le bouton **ON**.
- Le relais est activé.
- Le ventilateur démarre.

Cas 3 : Arrêt du ventilateur

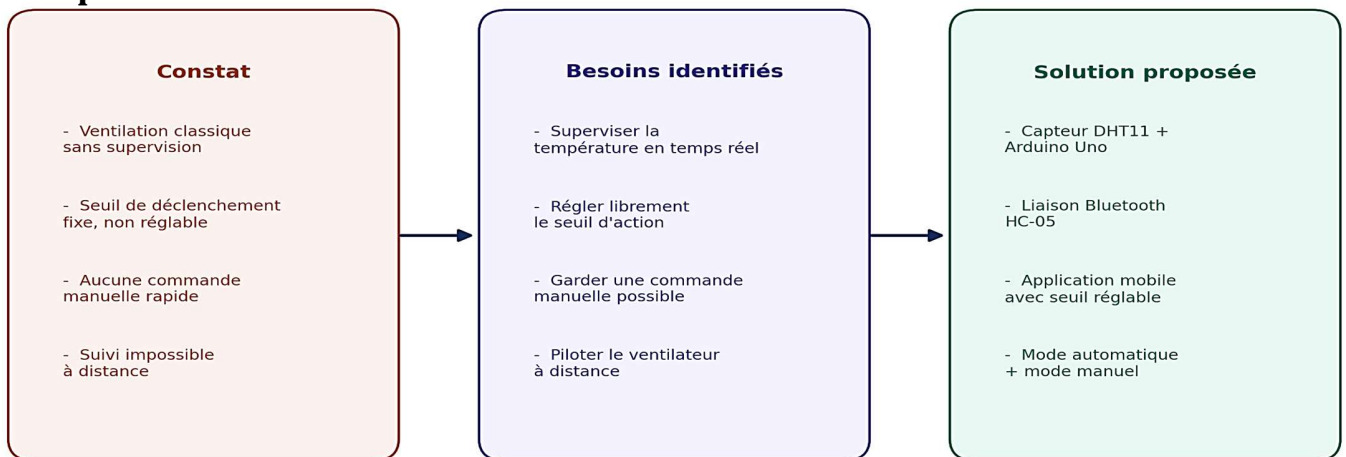
- L'utilisateur appuie sur le bouton **OFF**.
- Le relais se désactive.
- Le ventilateur s'arrête.

Cas 4 : Surveillance

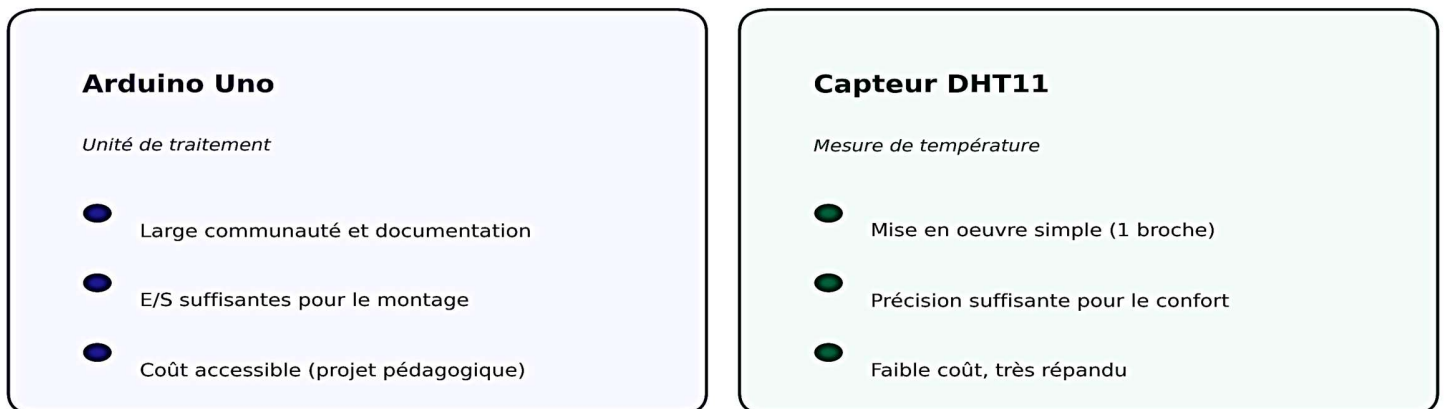
Pendant le fonctionnement, la température continue d'être affichée en temps réel.

8. Étapes de réalisation

Étape 1 : Étude du besoin.



Étape 2 : Choix des composants.



Module HC-05

Communication sans fil

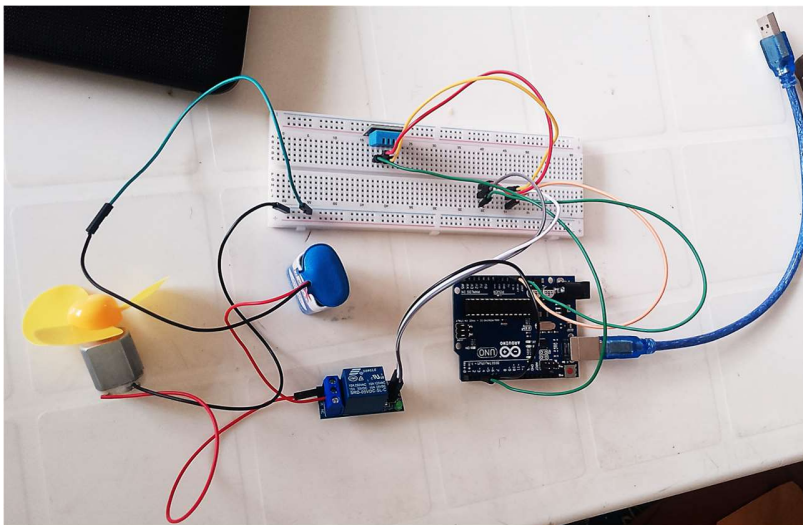
- Liaison série simple, sans réseau
- Compatible avec BluetoothClient
- Faible consommation, mise en oeuvre rapide

Module relais 1 canal

Commande de puissance

- Isole le circuit logique de puissance
- Pilotage sécurisé du ventilateur
- Commande simple par signal HIGH/LOW

Étape 3 : Montage sur breadboard.



Étape 4 : Programmation de l'Arduino.

CODE_FINAL

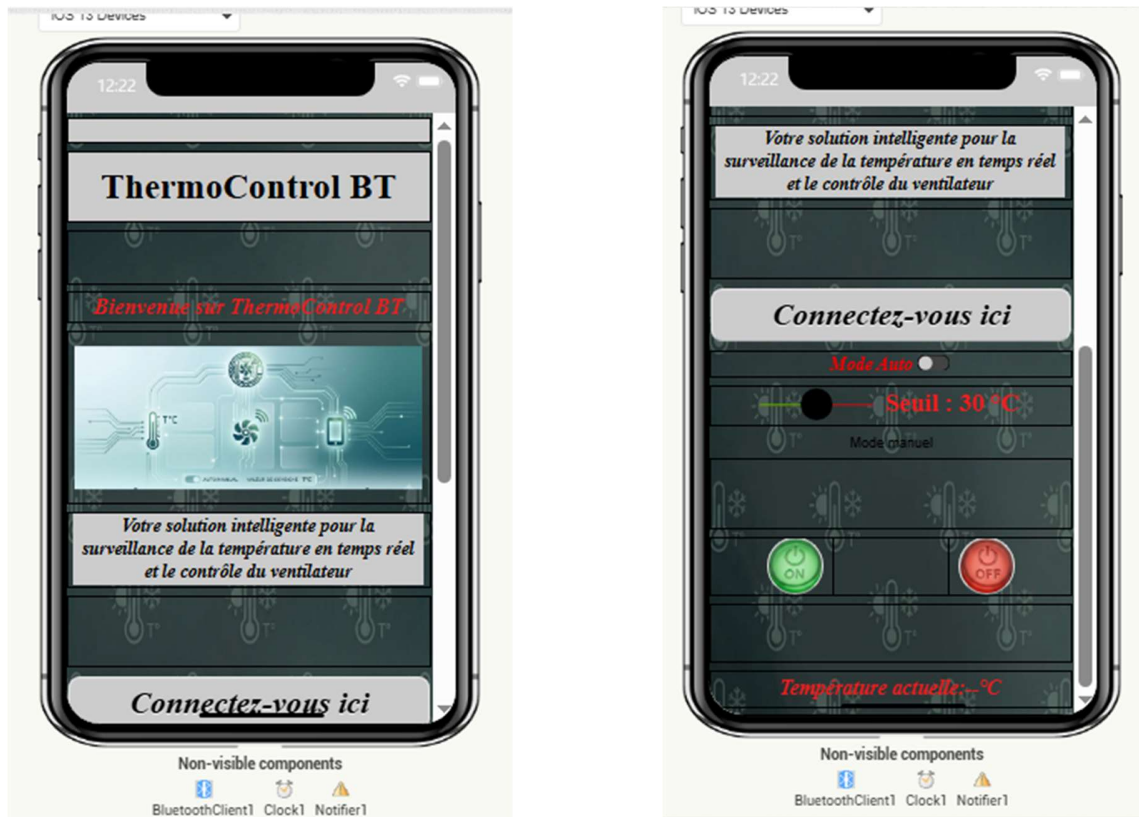
```
1 #include "DHT.h" //Bibliothèque du capteur DHT11
2 #include <SoftwareSerial.h> //Bibliothèque du module bluetooth
3
4 //Déclaration des variables
5 SoftwareSerial mySerial(10, 11); // RX, TX vers HC-05
6 const int DHTPIN = 2;
7 const int DHTTYPE = DHT11;
8 const int pinRelais = 3;
9 DHT dht(DHTPIN, DHTTYPE);
10 bool modeAuto;
11 float seuil;
12
13 unsigned long dernierEnvoi = 0;
14 const unsigned long intervalleEnvoi = 1000; // 1000 millisecondes, respecte le DHT11
15
16 //Initialisation des variables
17 void setup() {
18     pinMode(DHTPIN, INPUT);
19     pinMode(pinRelais, OUTPUT);
20     digitalWrite(pinRelais, HIGH); // état initial : ventilateur éteint
21     modeAuto = false;
22     seuil = 31.0; //Valeur par défaut de la température
23     dht.begin();
24     Serial.begin(9600);
```

```

4   Serial.begin(9600);
5   mySerial.begin(9600);
6 }
7
8
9 void loop() {
10  /* 1. Envoi périodique de la température sur le Bluetooth
11  //millis() renvoie le nombre de millisecondes écoulées depuis le démarrage de l'Arduino.
12  Pourquoi pas delay(1000) ? Parce que delay() bloquerait complètement
13  le programme pendant 1 seconde, empêchant l'Arduino de recevoir une commande "A" ou "B" pendant ce temps.
14  Avec millis(), l'Arduino reste réactif en permanence.
15  */
16  if (millis() - dernierEnvoi >= intervalleEnvoi) { /*Cette ligne vérifie : "est-ce que 2000 ms se sont écoulées depuis le dernier envoi ?" Si oui,
17                                                    on entre dans le bloc.*/
18
19      float t = dht.readTemperature();
20      if (!isnan(t)) {
21          mySerial.println(t); // envoie juste le nombre, ex: "25.30"
22          Serial.print("Température actuelle: ");
23          Serial.println(t); // pour debug sur le moniteur série USB
24          if (modeAuto){
25              if (t > seuil){
26                  digitalWrite(pinRelais, LOW);
27              }
28
29              else{
30                  digitalWrite(pinRelais, HIGH);
31              }
32          }
33          dernierEnvoi = millis();
34      }
35
36
37      // 2. Lecture des commandes manuelles reçues depuis l'app (A = ON, B = OFF)
38
39      if (mySerial.available()) {
40          String commande = mySerial.readStringUntil('\n');
41          commande.trim();
42
43          if (commande == "M1") {
44              modeAuto = true;
45          }
46          else if (commande == "M0") {
47              modeAuto = false;
48          }
49          else if (commande.startsWith("S")) {
50              seuil = commande.substring(1).toFloat(); // ex: "S28.5" -> 28.5
51
52              Serial.print("Nouveau seuil reçu");
53              Serial.println(seuil); // debug utile pour vérifier
54          }
55          else if (commande == "A" && !modeAuto) {
56              digitalWrite(pinRelais, LOW); // ventilateur ON
57              Serial.println("Commande A reçue -> Ventilateur ON"); // commande manuelle ON (uniquement si pas en auto)
58          }
59          else if (commande == "B" && !modeAuto) {
60              digitalWrite(pinRelais, HIGH); // ventilateur OFF
61              Serial.println("Commande B reçue -> Ventilateur OFF"); // commande manuelle OFF
62          }
63      }
64  }
65 }

```

Étape 5 : Développement de l'application mobile.

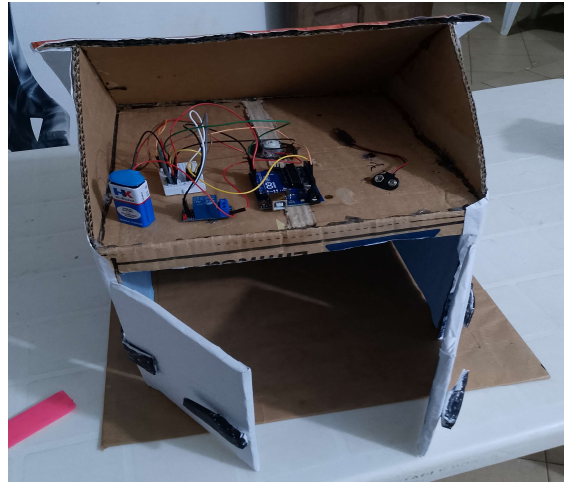


Étape 6 : Tests du système et validation du fonctionnement

Vérification des principales fonctions du régulateur thermique connecté

✓	Connexion Bluetooth L'application se connecte correctement au module HC-05	Validé
✓	Réception de la température La valeur affichée correspond à la mesure réelle du DHT11	Validé
✓	Réglage du seuil Le curseur modifie le seuil en temps réel, sans redémarrage	Validé
✓	Mode automatique Le ventilateur s'active au dépassement du seuil, s'arrête en dessous	Validé
✓	Mode manuel Les boutons Marche/Arrêt commandent directement le ventilateur	Validé
✓	Sécurité des modes Une alerte s'affiche si une commande manuelle est tentée en mode auto	Validé

Étape 7 : Réalisation de la maquette



Étape 8 : Présentation de la maquette finale



10. Conclusion

Notre projet a permis de réaliser un régulateur thermique connecté capable de mesurer la température, de transmettre les informations vers une application mobile et de commander manuellement un ventilateur via une liaison Bluetooth. Les objectifs fixés au départ ont été atteints. Le système est simple, économique, facile à utiliser et constitue une bonne introduction aux applications de l'Internet des Objets et de l'automatisation.

Résumé

Ce projet présente la conception et la réalisation d'un régulateur thermique connecté basé sur une carte Arduino Uno, un capteur DHT11, un module Bluetooth HC-05 et un relais 1 canal. La température est mesurée en temps réel puis affichée sur une application mobile développée avec MIT App Inventor. L'utilisateur peut également commander manuellement le ventilateur depuis son smartphone grâce à la communication Bluetooth. Ce système offre une solution pratique pour la surveillance de la température et le contrôle à distance d'un dispositif de ventilation.

Difficultés rencontrées

Au cours de la réalisation du projet, plusieurs difficultés ont été rencontrées :

- connexion Bluetooth entre le smartphone et le module HC-05 ;
- synchronisation de l'affichage de la température sur l'application ;
- alimentation du moteur CC ;
- débogage du programme Arduino ;
- tests de communication entre l'application et l'Arduino.

Ces difficultés ont été progressivement résolues grâce aux essais, aux corrections du programme et aux tests du montage.

Perspectives

Pour améliorer ce projet, plusieurs évolutions sont envisageables :

- automatiser complètement la mise en marche du ventilateur selon une température seuil ;
- ajouter un écran LCD ou OLED ;

- remplacer le Bluetooth par le Wi-Fi afin de permettre un contrôle via Internet ;
- enregistrer l'historique des températures ;
- envoyer des alertes lorsque la température devient trop élevée.

Ouverture

Ce projet peut être adapté à plusieurs domaines, notamment les habitations intelligentes (Smart Home), les salles serveurs, les laboratoires, les serres agricoles, les entrepôts de stockage ou encore les ateliers industriels. Il constitue une base solide pour le développement de systèmes de supervision plus avancés intégrant les technologies de l'Internet des Objets (IoT).